



PLATFORM AND AGENT HIGH-LEVEL DESIGN

Standard Agent Runtime

Abstract

Provides the development team with a standard runtime design, reusable class model, package structure, and implementation assumptions for Coaction agents.

Project Vega - Agentic Platform Ref Arch & Implementation

Table of Contents

TABLE OF CONTENTS	1
1. SCOPE	2
2. CURRENT DESIGN DECISIONS	2
3. TARGET RUNTIME ARCHITECTURE	3
4. RUNTIME HOSTING STRATEGY	3
5. STANDARD PYTHON PACKAGE STRUCTURE	4
6. CORE DOMAIN MODELS	5
7. EXECUTION PROFILE MODEL	6
8. RUNTIME ORCHESTRATION DESIGN	6
9. RETRIEVAL, MEMORY, AND TOOL GATEWAY ADAPTERS.....	8
9.1 BEDROCK KNOWLEDGE BASES RETRIEVAL.....	8
9.2 AGENTCORE MEMORY.....	8
9.3 AGENTCORE GATEWAY - READ-ONLY TOOLS.....	9
10. FASTAPI API LAYER.....	9
11. IDENTITY CONTEXT HANDLING	10
12. TELEMETRY AND AUDIT	10
13. FIRST-RELEASE TOOL ACTION SCOPE	11
14. REUSABLE AGENT TYPES.....	12
15. AWS CLIENT FACTORY.....	12
16. TESTING STRATEGY	12
17. DEVELOPER BUILD SEQUENCE	13
18. REMAINING DESIGN DECISION	13
APPENDIX A. REFERENCE SOURCES.....	14

1. Scope

This high-level design defines the reusable Python/FastAPI runtime foundation for Coaction agents. It translates the proposed reference architecture into developer-facing runtime contracts, package structure, domain models, base classes, adapter interfaces, and first-release implementation constraints.

Core design principle: every agent is configuration-driven. The runtime loads an approved execution profile and does not hardcode model IDs, knowledge bases, tool permissions, memory rules, prompt behavior, or observability behavior inside individual agent code.

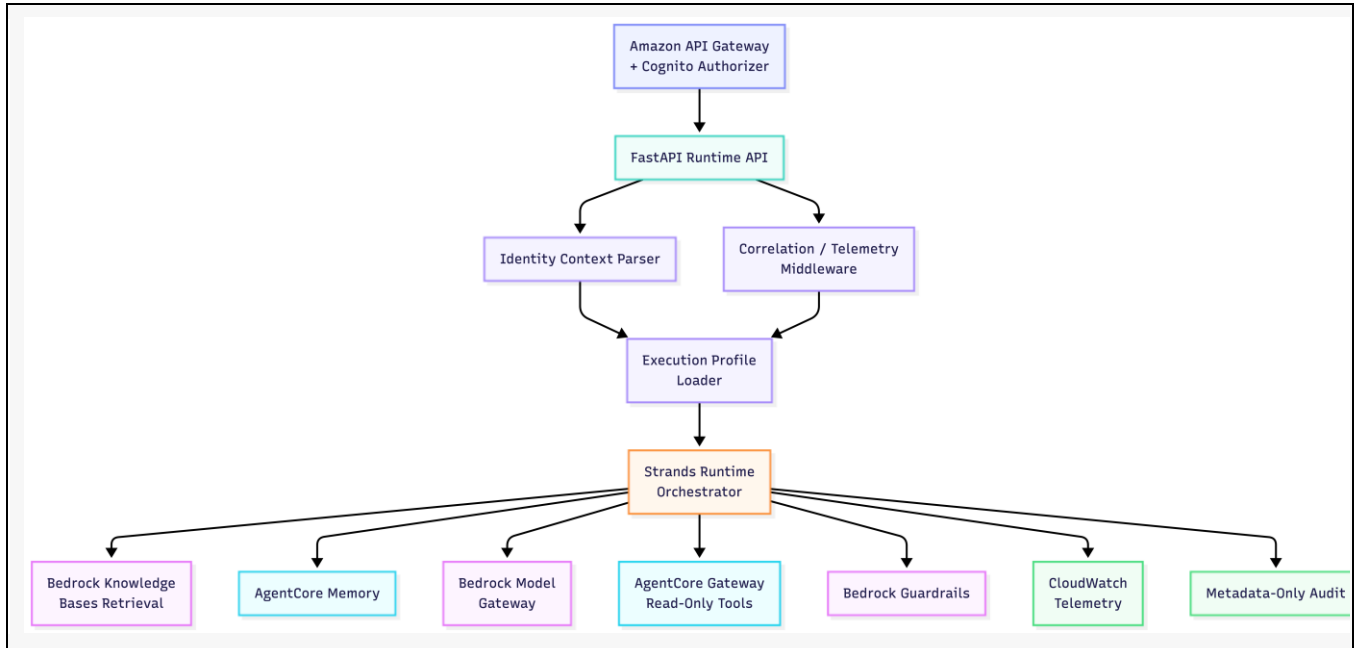
The design assumes FastAPI as the API/runtime service shell, Python as the implementation language, Strands Agents as the standard orchestration framework, and Boto3/AWS SDKs as the integration layer for AWS services.

2. Current Design Decisions

Area	Decision	Implementation impact
Runtime hosting	Not finalized	Build a hosting-neutral runtime. Use FastAPI as the API/runtime shell and isolate hosting behind RuntimeHostAdapter.
Orchestration framework	Strands Agents only	All agents extend the Strands-based base runtime. No alternate orchestration framework in the first implementation.
Authentication	Terminates at API Gateway	FastAPI consumes trusted identity context headers/claims and performs platform authorization and policy checks.
Retrieval	Amazon Bedrock Knowledge Bases only	Implement one concrete retrieval adapter: BedrockKnowledgeBaseRetriever.
Memory	Persistent memory is default for all agents	AgentCore Memory is part of the default execution path. Every agent has a memory profile, retention rule, and audit metadata.
Tool gateway	AgentCore Gateway is default	Implement AgentCoreToolGateway as the standard tool adapter.
Tool action scope	Read-only in first release	Tool registry and gateway allow only read actions initially. Workflow/write/external actions are blocked by policy.
Response schema	Assumed standard JSON envelope	Use a stable response envelope for portals, applications, and future channel adapters.
Observability	CloudWatch only	Implement CloudWatchTelemetryEmitter. No enterprise observability integration in the first release.
Payload logging	No raw prompt or response logging	Persist metadata, correlation IDs, source IDs, tool IDs, model IDs, status, and audit outcomes only.

3. Target Runtime Architecture

The target runtime uses API Gateway-authenticated requests, a FastAPI runtime shell, a Strands-based orchestrator, Bedrock Knowledge Bases for retrieval, AgentCore Memory for persistent memory, Bedrock model invocation, AgentCore Gateway for read-only tools, and CloudWatch for telemetry.



4. Runtime Hosting Strategy

Runtime hosting remains open. The implementation isolates hosting through a RuntimeHostAdapter so the development team can build the platform framework before finalizing whether runtime execution is hosted in ECS/Fargate, AgentCore Runtime, Lambda, or a hybrid pattern.

Hosting option	Fit
ECS/Fargate FastAPI service	Good default when API Gateway fronts the service and the team wants explicit API/runtime control.
Amazon Bedrock AgentCore Runtime	Good fit when the agent runtime is deployed as an AgentCore-hosted endpoint.
AWS Lambda	Useful for lightweight APIs, but less ideal for longer-lived, streaming, or session-heavy agent flows.
Hybrid	FastAPI API facade with selected agents hosted in AgentCore Runtime.

```

from abc import ABC, abstractmethod
from typing import Any

class RuntimeHostAdapter(ABC):
    @abstractmethod
    async def invoke(self, payload: dict[str, Any]) -> dict[str, Any]:
        """Invoke the runtime host."""
        raise NotImplementedError
  
```

```

class LocalFastApiRuntimeHost(RuntimeHostAdapter):
    async def invoke(self, payload: dict[str, Any]) -> dict[str, Any]:
        # Runs orchestration locally inside the FastAPI service.
        ...

class AgentCoreRuntimeHost(RuntimeHostAdapter):
    async def invoke(self, payload: dict[str, Any]) -> dict[str, Any]:
        # Invokes Amazon Bedrock AgentCore Runtime when selected.
        ...

```

5. Standard Python Package Structure

The package structure separates API routing, domain models, runtime orchestration, control-plane repositories, service abstractions, AWS adapters, and reusable agent templates.

```

coaction_agent_platform/
|
|-- app/
|   |-- main.py
|   |-- routers/
|   |   |-- invoke.py
|   |   |-- sessions.py
|   |   |-- feedback.py
|   |   |-- health.py
|   |-- dependencies/
|   |   |-- identity.py
|   |   |-- services.py
|   |   |-- settings.py
|   |-- middleware/
|   |   |-- correlation.py
|   |   |-- telemetry.py
|   |   |-- errors.py
|
|-- domain/
|   |-- invocation.py
|   |-- identity.py
|   |-- execution_profile.py
|   |-- retrieval.py
|   |-- memory.py
|   |-- tools.py
|   |-- response.py
|   |-- audit.py
|
|-- runtime/
|   |-- base_agent.py
|   |-- strands_agent.py
|   |-- orchestrator.py
|   |-- context_builder.py
|   |-- response_composer.py
|   |-- host_adapter.py
|
|-- control_plane/
|   |-- agent_registry.py
|   |-- execution_profile_repository.py
|   |-- prompt_repository.py
|   |-- policy_repository.py
|   |-- tool_registry.py
|
|-- services/
|   |-- authorization.py
|   |-- guardrails.py
|   |-- retrieval.py
|   |-- memory.py
|   |-- model_gateway.py
|   |-- tool_gateway.py
|   |-- telemetry.py
|   |-- audit.py
|
|-- adapters/
|   |-- aws/
|   |   |-- boto3_factory.py
|   |   |-- dynamodb.py
|   |   |-- s3.py

```

```

| | |-- secrets_manager.py
| | |-- bedrock_model.py
| | |-- bedrock_kb.py
| | |-- bedrock_guardrails.py
| | |-- agentcore_gateway.py
| | |-- agentcore_memory.py
| | |-- cloudwatch.py
| | |-- eventbridge.py
| |-- enterprise/
| |-- readonly_api_adapter.py
|
|-- agents/
| |-- readonly_agent.py
| |-- retrieval_agent.py
| |-- templates.py
|
|-- tests/
| |-- unit/
| |-- integration/
| |-- regression/

```

6. Core Domain Models

Pydantic models define the shared API and runtime contracts. These models keep agent invocation, identity context, citations, tool results, and responses consistent across agents.

```

from pydantic import BaseModel, Field
from typing import Any, Literal

class AgentInvocationRequest(BaseModel):
    agent_id: str
    input_text: str
    session_id: str | None = None
    channel: str = "api"
    request_metadata: dict[str, Any] = Field(default_factory=dict)

class IdentityContext(BaseModel):
    user_id: str
    roles: list[str] = Field(default_factory=list)
    channel: str
    application_id: str | None = None
    session_id: str | None = None
    correlation_id: str
    claims: dict[str, Any] = Field(default_factory=dict)

class SourceCitation(BaseModel):
    source_id: str
    title: str | None = None
    uri: str | None = None
    chunk_id: str | None = None
    score: float | None = None

class ToolResult(BaseModel):
    tool_id: str
    action_class: Literal["read"]
    status: Literal["success", "failed", "blocked"]
    result_summary: str | None = None
    error_code: str | None = None

class AgentInvocationResponse(BaseModel):
    status: Literal[
        "success", "clarification_required", "blocked", "escalated", "error"
    ]
    answer: str
    citations: list[SourceCitation] = Field(default_factory=list)
    tool_results: list[ToolResult] = Field(default_factory=list)
    session_id: str
    correlation_id: str
    model_id: str | None = None
    metadata: dict[str, Any] = Field(default_factory=dict)

```

7. Execution Profile Model

The execution profile is the runtime source of truth. It defines the model, retrieval, memory, tool, guardrail, observability, and response contract used for a specific agent version.

```
class ModelProfile(BaseModel):
    provider: Literal["bedrock"] = "bedrock"
    model_id: str
    temperature: float = 0.0
    max_tokens: int | None = None
    fallback_model_id: str | None = None

class RetrievalProfile(BaseModel):
    provider: Literal["bedrock_knowledge_base"] = "bedrock_knowledge_base"
    enabled: bool = True
    knowledge_base_ids: list[str]
    metadata_filters: dict[str, Any] = Field(default_factory=dict)
    reranking_enabled: bool = True
    min_confidence: float | None = None
    citations_required: bool = True

class MemoryProfile(BaseModel):
    provider: Literal["agentcore_memory"] = "agentcore_memory"
    enabled: bool = True
    persistent: bool = True
    memory_scope: Literal["agent_user", "agent_session", "agent_task"] = "agent_user"
    retention_days: int = 90
    read_enabled: bool = True
    write_enabled: bool = True

class ToolPermission(BaseModel):
    tool_id: str
    action_class: Literal["read"] = "read"
    allowed_roles: list[str] = Field(default_factory=list)
    requires_approval: bool = False

class GuardrailProfile(BaseModel):
    guardrail_id: str | None = None
    guardrail_version: str | None = None
    input_check_enabled: bool = True
    output_check_enabled: bool = True

class ObservabilityProfile(BaseModel):
    provider: Literal["cloudwatch"] = "cloudwatch"
    emit_metrics: bool = True
    emit_traces: bool = True
    log_raw_prompt: bool = False
    log_raw_response: bool = False

class ExecutionProfile(BaseModel):
    agent_id: str
    version: str
    orchestration_framework: Literal["strands"] = "strands"
    prompt_template_id: str
    model_profile: ModelProfile
    retrieval_profile: RetrievalProfile
    memory_profile: MemoryProfile
    tool_permissions: list[ToolPermission] = Field(default_factory=list)
    guardrail_profile: GuardrailProfile
    observability_profile: ObservabilityProfile = ObservabilityProfile()
    response_contract_version: str = "v1"
```

8. Runtime Orchestration Design

All agents extend a Strands-based base agent. The base agent delegates execution to the runtime orchestrator, which enforces the standard sequence for every invocation.

```
from abc import ABC, abstractmethod
```

```

class BaseAgent(ABC):
    def __init__(self, agent_id: str, orchestrator: "RuntimeOrchestrator") -> None:
        self.agent_id = agent_id
        self.orchestrator = orchestrator

    async def invoke(
        self,
        request: AgentInvocationRequest,
        identity: IdentityContext,
    ) -> AgentInvocationResponse:
        return await self.orchestrator.execute(request, identity)

    @abstractmethod
    def agent_type(self) -> str:
        raise NotImplementedError

class StrandsBaseAgent(BaseAgent):
    def agent_type(self) -> str:
        return "strands_agent"

    async def build_strands_context(self, runtime_context: dict) -> dict:
        return runtime_context

```

```

class RuntimeOrchestrator:
    def __init__(
        self,
        agent_registry: "AgentRegistryRepository",
        profile_repo: "ExecutionProfileRepository",
        authorization: "AuthorizationService",
        guardrails: "GuardrailService",
        retriever: "BedrockKnowledgeBaseRetriever",
        memory: "AgentCoreMemoryProvider",
        model_gateway: "BedrockModelGateway",
        tool_gateway: "AgentCoreReadOnlyToolGateway",
        response_composer: "ResponseComposer",
        telemetry: "CloudWatchTelemetryEmitter",
        audit: "MetadataOnlyAuditLogger",
    ) -> None:
        self.agent_registry = agent_registry
        self.profile_repo = profile_repo
        self.authorization = authorization
        self.guardrails = guardrails
        self.retriever = retriever
        self.memory = memory
        self.model_gateway = model_gateway
        self.tool_gateway = tool_gateway
        self.response_composer = response_composer
        self.telemetry = telemetry
        self.audit = audit

    async def execute(
        self,
        request: AgentInvocationRequest,
        identity: IdentityContext,
    ) -> AgentInvocationResponse:
        agent = await self.agent_registry.get_active_agent(request.agent_id)
        profile = await self.profile_repo.get_profile(
            agent_id=request.agent_id,
            version=agent.active_version,
        )

        await self.authorization.authorize_invocation(identity, agent, profile)
        await self.guardrails.check_input(request, profile)

        memory_context = await self.memory.read(request, identity, profile)
        retrieved_context = await self.retriever.retrieve(request, identity, profile)

        model_result = await self.model_gateway.invoke(
            request=request,
            identity=identity,
            profile=profile,

```

```

        memory_context=memory_context,
        retrieved_context=retrieved_context,
    )

    tool_results = await self.tool_gateway.execute_readonly_tools(
        model_result=model_result,
        identity=identity,
        profile=profile,
    )

    response = await self.response_composer.compose(
        request=request,
        profile=profile,
        model_result=model_result,
        retrieved_context=retrieved_context,
        memory_context=memory_context,
        tool_results=tool_results,
    )

    await self.guardrails.check_output(response, profile)
    await self.memory.write(request, response, identity, profile)
    await self.telemetry.emit_invocation(request, response, profile)
    await self.audit.record_invocation(request, response, identity, profile)
    return response

```

9. Retrieval, Memory, and Tool Gateway Adapters

9.1 Bedrock Knowledge Bases Retrieval

```

class BedrockKnowledgeBaseRetriever:
    def __init__(self, boto3_factory: "Boto3SessionFactory") -> None:
        self.client = boto3_factory.client("bedrock-agent-runtime")

    async def retrieve(
        self,
        request: AgentInvocationRequest,
        identity: IdentityContext,
        profile: ExecutionProfile,
    ) -> list[SourceCitation]:
        # Use profile.retrieval_profile.knowledge_base_ids.
        # Apply metadata filters and return citation metadata.
        ...

```

9.2 AgentCore Memory

```

class AgentCoreMemoryProvider:
    def __init__(self, boto3_factory: "Boto3SessionFactory") -> None:
        self.client = boto3_factory.client("bedrock-agentcore")

    async def read(
        self,
        request: AgentInvocationRequest,
        identity: IdentityContext,
        profile: ExecutionProfile,
    ) -> dict:
        if not profile.memory_profile.enabled:
            return {}
        # Read scoped persistent memory for this agent/user/session/task.
        ...

    async def write(
        self,
        request: AgentInvocationRequest,
        response: AgentInvocationResponse,
        identity: IdentityContext,
        profile: ExecutionProfile,
    ) -> None:

```

```

if not profile.memory_profile.enabled or not profile.memory_profile.write_enabled:
    return
# Write memory only after applying data minimization rules.
...

```

9.3 AgentCore Gateway - Read-Only Tools

```

class AgentCoreReadOnlyToolGateway:
    def __init__(self, boto3_factory: "Boto3SessionFactory") -> None:
        self.client = boto3_factory.client("bedrock-agentcore")

    async def execute_readonly_tools(
        self,
        model_result: dict,
        identity: IdentityContext,
        profile: ExecutionProfile,
    ) -> list[ToolResult]:
        requested_tools = self._extract_requested_tools(model_result)
        results: list[ToolResult] = []

        for tool_request in requested_tools:
            permission = self._resolve_permission(tool_request.tool_id, profile)
            if permission.action_class != "read":
                results.append(
                    ToolResult(
                        tool_id=tool_request.tool_id,
                        action_class="read",
                        status="blocked",
                        error_code="NON_READ_ACTION_BLOCKED",
                    )
                )
                continue
            # Invoke through AgentCore Gateway.
            ...
        return results

```

10. FastAPI API Layer

The initial API surface exposes a small set of runtime endpoints. Additional admin/control-plane endpoints are not exposed to normal runtime users.

Endpoint	Purpose
POST /v1/agents/{agent_id}/invoke	Invoke an agent.
GET /v1/agents/{agent_id}	Read agent metadata.
GET /v1/sessions/{session_id}	Read session metadata.
DELETE /v1/sessions/{session_id}	Clear or close a session.
POST /v1/feedback	Capture user feedback.
GET /health	Liveness check.
GET /ready	Dependency readiness check.

```

from fastapi import APIRouter, Depends

```

```
router = APIRouter(prefix="/v1/agents", tags=["agent-runtime"])

@router.post("/{agent_id}/invoke", response_model=AgentInvocationResponse)
async def invoke_agent(
    agent_id: str,
    request: AgentInvocationRequest,
    identity: IdentityContext = Depends(get_identity_context),
    orchestrator: RuntimeOrchestrator = Depends(get_runtime_orchestrator),
) -> AgentInvocationResponse:
    request.agent_id = agent_id
    return await orchestrator.execute(request, identity)
```

11. Identity Context Handling

Primary authentication terminates at API Gateway. The FastAPI layer parses validated identity context from gateway/authorizer headers and performs platform authorization.

Header	Purpose
X-User-Id	Validated user identifier.
X-Roles	Comma-separated role or entitlement list.
X-Channel	Calling channel or consumer type.
X-Correlation-Id	End-to-end correlation identifier.
X-Session-Id	Optional session identifier.

```
from fastapi import Header, HTTPException

async def get_identity_context(
    x_user_id: str = Header(..., alias="X-User-Id"),
    x_roles: str = Header("", alias="X-Roles"),
    x_channel: str = Header("api", alias="X-Channel"),
    x_correlation_id: str = Header(..., alias="X-Correlation-Id"),
) -> IdentityContext:
    if not x_user_id:
        raise HTTPException(status_code=401, detail="Missing identity context")

    return IdentityContext(
        user_id=x_user_id,
        roles=[r.strip() for r in x_roles.split(",") if r.strip()],
        channel=x_channel,
        correlation_id=x_correlation_id,
    )
```

12. Telemetry and Audit

The first release uses CloudWatch-only observability and metadata-only audit logging. Raw prompts and raw responses are not logged by default.

Telemetry / audit field	Description
correlation_id	End-to-end request correlation.

Telemetry / audit field	Description
agent_id / agent_version	Agent identity and configuration version.
user_id / channel	Validated caller metadata.
status	Runtime outcome.
model_id	Model invoked.
citation_count	Number of returned citations.
tool_count	Number of tool calls.
raw_prompt_logged	Always false by default.
raw_response_logged	Always false by default.

```

class MetadataOnlyAuditLogger:
    async def record_invocation(
        self,
        request: AgentInvocationRequest,
        response: AgentInvocationResponse,
        identity: IdentityContext,
        profile: ExecutionProfile,
    ) -> None:
        audit_event = {
            "correlation_id": identity.correlation_id,
            "agent_id": request.agent_id,
            "agent_version": profile.version,
            "user_id": identity.user_id,
            "channel": identity.channel,
            "status": response.status,
            "model_id": response.model_id,
            "citation_count": len(response.citations),
            "tool_count": len(response.tool_results),
            "raw_prompt_logged": False,
            "raw_response_logged": False,
        }
        ...

```

13. First-Release Tool Action Scope

Action class	First-release support	Runtime behavior
Read-only lookup	Yes	Allowed through AgentCore Gateway when agent, user, and tool policy permit.
Document processing	Limited	Allowed only if it does not mutate enterprise systems.
Workflow initiation	No	Blocked by policy.
Write/update	No	Blocked by policy.
External interaction	No	Blocked by policy.

14. Reusable Agent Types

The first release needs two reusable agent templates: a retrieval agent and a read-only tool agent.

```
class RetrievalAgent(StrandsBaseAgent):
    def agent_type(self) -> str:
        return "retrieval_agent"

class ReadOnlyToolAgent(StrandsBaseAgent):
    def agent_type(self) -> str:
        return "readonly_tool_agent"
```

Agent type	Default behavior
RetrievalAgent	Bedrock KB retrieval enabled, citations enabled, AgentCore Memory enabled, tools disabled or optional read-only.
ReadOnlyToolAgent	Bedrock KB retrieval enabled, AgentCore Memory enabled, read-only AgentCore Gateway tools enabled, workflow/write/external actions blocked.

15. AWS Client Factory

All AWS adapters use one Boto3 client factory. This prevents each agent or adapter from independently creating clients and makes retry, timeout, and region behavior consistent.

```
import boto3
from botocore.config import Config

class Boto3SessionFactory:
    def __init__(self, region_name: str) -> None:
        self.region_name = region_name
        self.config = Config(
            retries={"max_attempts": 3, "mode": "standard"},
            connect_timeout=3,
            read_timeout=30,
        )

    def client(self, service_name: str):
        return boto3.client(
            service_name,
            region_name=self.region_name,
            config=self.config,
        )
```

16. Testing Strategy

Test category	Scope
Unit tests	AuthorizationService, ExecutionProfileRepository, ResponseComposer, MemoryProvider, ToolGateway, Retriever.
Contract tests	Execution profile schema, tool input/output schemas, response envelope, retrieval result schema.
Integration tests	Bedrock model invocation, Bedrock Knowledge Bases retrieval, AgentCore Memory, AgentCore Gateway, CloudWatch telemetry.

Test category	Scope
Agent regression tests	Known prompts, expected retrieval behavior, citations, blocked prompts, low-confidence behavior, read-only tool calls.

17. Developer Build Sequence

1. Build FastAPI shell, middleware, and routers.
2. Implement API Gateway identity-context parsing.
3. Implement domain models and response envelope.
4. Implement Boto3 client factory.
5. Implement agent registry and execution profile repositories.
6. Implement Strands-only base agent.
7. Implement runtime orchestrator.
8. Implement Bedrock model gateway.
9. Implement Bedrock Knowledge Bases retriever.
10. Implement AgentCore Memory provider as default.
11. Implement AgentCore Gateway read-only tool adapter.
12. Implement CloudWatch telemetry emitter.
13. Implement metadata-only audit logger.
14. Implement retrieval agent template.
15. Implement read-only tool agent template.
16. Implement regression test harness.
17. Package runtime for the selected hosting target once hosting is decided.

18. Remaining Design Decision

Decision	Impact
Runtime hosting: ECS/Fargate FastAPI service vs AgentCore Runtime vs hybrid	Determines container packaging, invocation path, CI/CD deployment target, scaling model, streaming behavior, and runtime IAM model.

The current code design isolates the hosting decision through RuntimeHostAdapter, so the development team can start the reusable platform framework before final hosting is finalized.

Appendix A. Reference Sources

The following sources support the framework and AWS service roles referenced in this HLD.

FastAPI APIRouter and dependency model: <https://fastapi.tiangolo.com/reference/apirouter/>

FastAPI Dependencies: <https://fastapi.tiangolo.com/tutorial/dependencies/>

Boto3 - AWS SDK for Python: <https://docs.aws.amazon.com/boto3/latest/>

Amazon Bedrock AgentCore Gateway: <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway.html>

Amazon Bedrock AgentCore Memory: <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/memory.html>

Amazon Bedrock AgentCore Runtime invocation: https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_InvokeAgentRuntime.html

Amazon Bedrock Knowledge Bases: <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html>

Amazon CloudWatch Generative AI Observability:
<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/GenAI-observability.html>